Spaceship Titanic — End-to-End ML Learning Report

From raw messy data to a tuned, evaluated, submitted classification model

Built and explained step-by-step as a hands-on beginner project

Kaggle Playground Series · Binary Classification · XGBoost Final Model · 78.9% Validation Accuracy

Table of Contents

1. The Problem — What Are We Actually Solving?

The question: given a passenger's info (age, spending, cabin, home planet...), was this passenger `Transported` to an alternate dimension during the Spaceship Titanic's collision with a spacetime anomaly? Answer: **True** or **False**.

This makes it a **binary classification** problem — not regression (we're not predicting a number like a price), and not multi-class (only two possible outcomes). That single fact decides which family of models is even eligible: Linear Regression is out; Logistic Regression, Random Forest Classifier, and XGBoost Classifier are in.

Why this matters before writing any code

Framing the problem correctly first is what separates someone who copies tutorial code from someone who understands what they're doing. Three questions to always answer before touching data:

Question	Answer for this project
What am I predicting?	`Transported` — True/False → Classification
What's the input?	Mixed categorical + numeric passenger data
How am I scored?	Accuracy (Kaggle grades this competition on plain accuracy)

Why accuracy is a trustworthy metric here: it only works cleanly when classes are roughly balanced. We verified this early — Transported was ~50.4% True / 49.6% False. With a skewed target (like fraud detection, where <1% of cases are positive), accuracy alone would be misleading, and we'd need precision/recall/ROC-AUC instead.

2. Understanding the Raw Data

8,693 rows, 14 columns. Before cleaning anything, we read the dataset like an engineer, not a spectator — grouping columns by what they actually represent:

Column(s)	What it is	Cleaning need
PassengerId	Format `gggg_pp` — group + position in group	Extract group number, then drop
HomePlanet, Destination	Categorical (Earth/Europa/Mars, 3 destinations)	Fill missing, one-hot encode
CryoSleep, VIP	Boolean, but stored as text due to missing values	Fill missing, convert to real bool
Cabin	Format `deck/num/side` packed into one string	Split into 3 columns
Age, RoomService...VRDeck	Numeric (age, 5 spending categories)	Fill missing with logic/median
Name	Free text	Dropped — low direct signal

Why check dtypes and missing counts before anything else: running `train_df.info()` and `train_df.isna().sum()` first told us every feature had roughly 180-220 missing values (~2%) — evenly spread, nothing catastrophic, nothing biased toward one class. That single check shaped the entire cleaning strategy: small, fixable gaps, not a broken dataset.

```
train_df = pd.read_csv('/kaggle/input/competitions/spaceship-titanic/train.csv')
train_df.info()
train_df.isna().sum()
train_df['Transported'].value_counts(normalize=True)  # check class balance
```

3. Data Cleaning — Logic Over Guessing

The single biggest lesson of this project: **filling missing values isn't a mechanical step — it's a place to apply domain logic.** Blindly averaging or defaulting to "Unknown" throws away information that's often recoverable if you think about what the data actually represents.

3.1 CryoSleep — domain-informed imputation

> **The insight:** a passenger in CryoSleep is asleep for the whole voyage — they physically cannot spend money. So if a passenger's total spending across all 5 categories is exactly zero, they were almost certainly asleep. If they spent anything, they were awake.

```
spend_cols = ['RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']
train_df['Total Spend'] = train_df[spend_cols].sum(axis=1)

missing_mask = train_df['CryoSleep'].isna()
train_df.loc[missing_mask, 'CryoSleep'] = train_df.loc[missing_mask, 'Total Spend'] == 0
```

Why not a for-loop? Pandas is built around **vectorized operations** — applying logic to an entire column at once instead of row-by-row. `.loc[boolean_mask, column]` reads as "give me this column, but only for rows where the mask is True" — filtering and assignment in one step, dramatically faster than iterating manually, and it's the idiomatic way real pandas code is written.

> **Validation before applying:** we checked `.describe()` on the TotalSpend of the 217 missing-CryoSleep rows first. Result: 98 rows had exactly 0 spend, 119 had varied nonzero spend — a real, meaningful split, confirming the rule would actually differentiate rows rather than being arbitrary.

3.2 VIP — mode fallback (no logical rule available)

Unlike CryoSleep, there's no clean logical link between VIP status and any other column — spending a lot doesn't necessarily mean someone bought VIP status. When no domain rule exists, the standard fallback is the **mode** (most common value):

```
train_df['VIP'] = train_df['VIP'].fillna(False)  # False was overwhelmingly the mode (8291 vs 199)
```

> **Why mode, not "Unknown":** VIP status is rare to begin with — defaulting unknowns to "not VIP" is a low-risk, statistically sound assumption. Adding a third "Unknown" category would add complexity for a column where the missing rate is only ~2%, with little benefit.

3.3 Cabin — split, then group-based recovery, then mode fallback

Cabin values look like `B/0/P` — Deck, CabinNum, and Side packed into one string.

```
train_df[['Deck', 'CabinNum', 'Side']] = train_df['Cabin'].str.split('/', expand=True)
```

> **Why `expand=True` matters:** without it, `.str.split('/')` returns one column where every cell holds a Python list, e.g. `['B','0','P']` — still unusable by a model. `expand=True` spreads those list items into three real columns.

For the 199 missing Cabin values, we used a two-tier recovery strategy instead of guessing directly:

1. **Tier 1 — group-mates:** Passengers traveling together (same group number extracted from PassengerId) often share a cabin. We checked: does this passenger's group have *any other member* with a known cabin? If so, borrow it.
2. **Tier 2 — mode fallback:** For passengers with no group-mate to borrow from (mostly solo travelers), fall back to the most common Deck (`'F'`) and Side (`'S'`).

```
cabin_by_group = train_df[train_df['Cabin'].notna()].groupby('Group')['Cabin'].first()
train_df['Cabin'] = train_df.apply(
    lambda row: cabin_by_group.get(row['Group'], row['Cabin']) if pd.isna(row['Cabin']) else row['Cabin'],
    axis=1
)
# Result: 100 of 199 recovered via group-mates; remaining 99 → mode fallback
train_df['Deck']  = train_df['Deck'].fillna('F')
train_df['Side']  = train_df['Side'].fillna('S')
```

> **Why check group sizes first:** `value_counts().value_counts()` on the group column showed ~4,805 groups of size 1 (solo travelers) versus ~3,888 groups of size 2+. Knowing over half the population travels solo told us upfront that the group-recovery trick would only partially cover the gap — so we built the mode fallback deliberately, not as an afterthought.

3.4 Age — median, not mean

```
train_df['Age'] = train_df['Age'].fillna(train_df['Age'].median())
```

> **Why median over mean:** Age had mean 28.83 vs median 27.0 — a mild rightward skew. Median represents the "typical" passenger regardless of outliers pulling the average up; it's the defensible, standard choice for numeric imputation with skewed data.

3.5 Spending columns — CryoSleep logic first, then median

```
for col in spend_cols:
    # CryoSleep passengers cannot spend anything — fill their gaps with 0 first
    train_df.loc[train_df['CryoSleep'] == True, col] = train_df.loc[train_df['CryoSleep'] == True, col].fillna(0)
    # Remaining gaps (awake passengers, unknown spend) → median
    train_df[col] = train_df[col].fillna(train_df[col].median())
```

3.6 HomePlanet / Destination — mode fallback

```
train_df['HomePlanet']  = train_df['HomePlanet'].fillna('Earth')          # most common: 4602/8492
train_df['Destination'] = train_df['Destination'].fillna('TRAPPIST-1e')   # most common: 5915/8511
```

3.7 Dropping low-signal columns

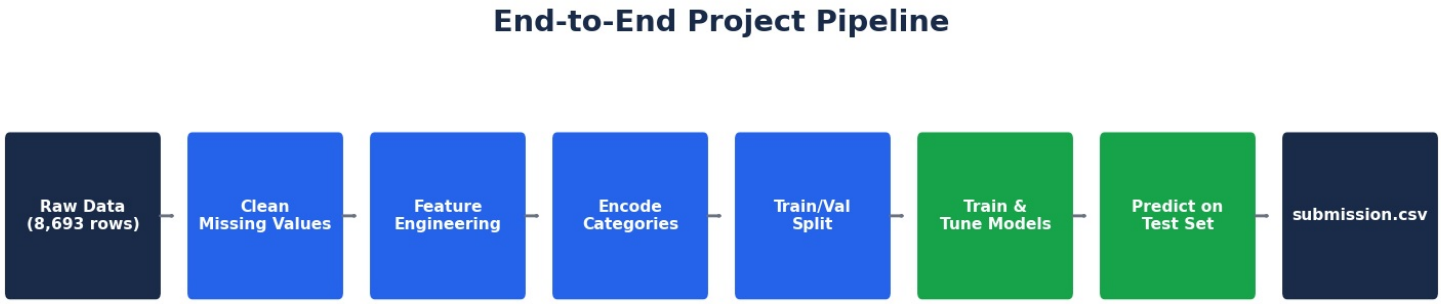
```
train_df = train_df.drop(columns=['Name', 'Cabin', 'CabinNum', 'PassengerId', 'Group'])
```

Column dropped	Reason
Name	Free text, no direct numeric signal for a first model
Cabin	Already extracted into Deck / CabinNum / Side — redundant now
CabinNum	Raw cabin number carries little standalone predictive meaning
PassengerId	Unique per row — a model would "memorize" instead of learning a pattern
Group	Only needed temporarily to recover Cabin — derived, not independent signal

4. Feature Engineering

Feature engineering means creating new, more useful columns from the raw ones. In this project:

- **Deck, CabinNum, Side** — extracted from the packed Cabin string
- **Group** — extracted from PassengerId, used temporarily to recover missing Cabins
- **Total Spend** — sum of all 5 spending columns, used both as a CryoSleep-inference signal and later evaluated as a feature (and eventually dropped from the model input — see Section 8.2)



5. Encoding Categories to Numbers

Models only understand numbers. Two different techniques were used, deliberately chosen based on the type of category:

5.1 Boolean columns → 0 / 1

```
train_df['CryoSleep']   = train_df['CryoSleep'].astype(int)
train_df['VIP']         = train_df['VIP'].astype(int)
train_df['Transported'] = train_df['Transported'].astype(int)
```

True/False has a natural numeric mapping — no fake ordering implied.

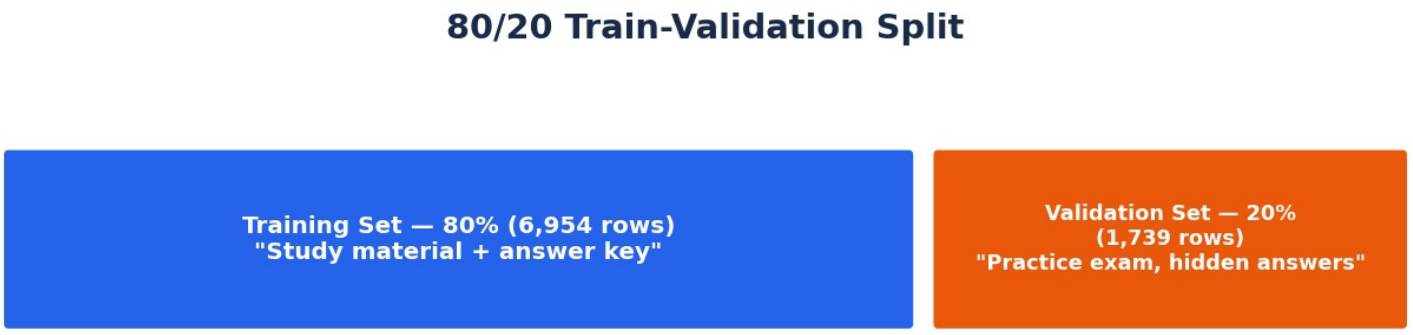
5.2 Multi-category columns → One-Hot Encoding

```
train_df = pd.get_dummies(train_df, columns=['HomePlanet', 'Destination', 'Deck', 'Side'], drop_first=True)
```

Why NOT simple label encoding (0, 1, 2, 3...) here: labeling Earth=0, Europa=1, Mars=2 would falsely imply Mars > Europa > Earth in some meaningful numeric sense. One-hot encoding instead creates a separate True/False column per category (e.g. `HomePlanet_Europa`, `HomePlanet_Mars`) — no fake ranking, just "is it this category or not."

Why `drop_first=True`: drops one category per column to avoid redundant information (the "dummy variable trap") — if a row is 0 for both Europa and Mars, the model already knows it must be Earth. This matters most for linear models like Logistic Regression, where redundant columns can cause unstable, hard-to-interpret coefficients; it costs nothing for tree-based models like Random Forest/XGBoost, so it's the safe default either way.

6. Train / Validation Split




```
from sklearn.model_selection import train_test_split
X = train_df.drop(columns=['Transported'])
y = train_df['Transported']
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
```

Why this exists at all: a model tested on data it already trained on will always look artificially good — like grading a student on the exact questions they memorized answers to. Holding back 20% the model never sees during training gives a trustworthy estimate of real-world performance.

Split ratio	Trade-off
50/50	Too little training data — weaker model, especially on a dataset this size (8,693 rows)
95/5 or 90/10	Validation set too small — accuracy score becomes noisy / unreliable, bounces with luck
80/20 (used)	Standard balance for small-to-medium datasets — enough to train on, enough to trust the score

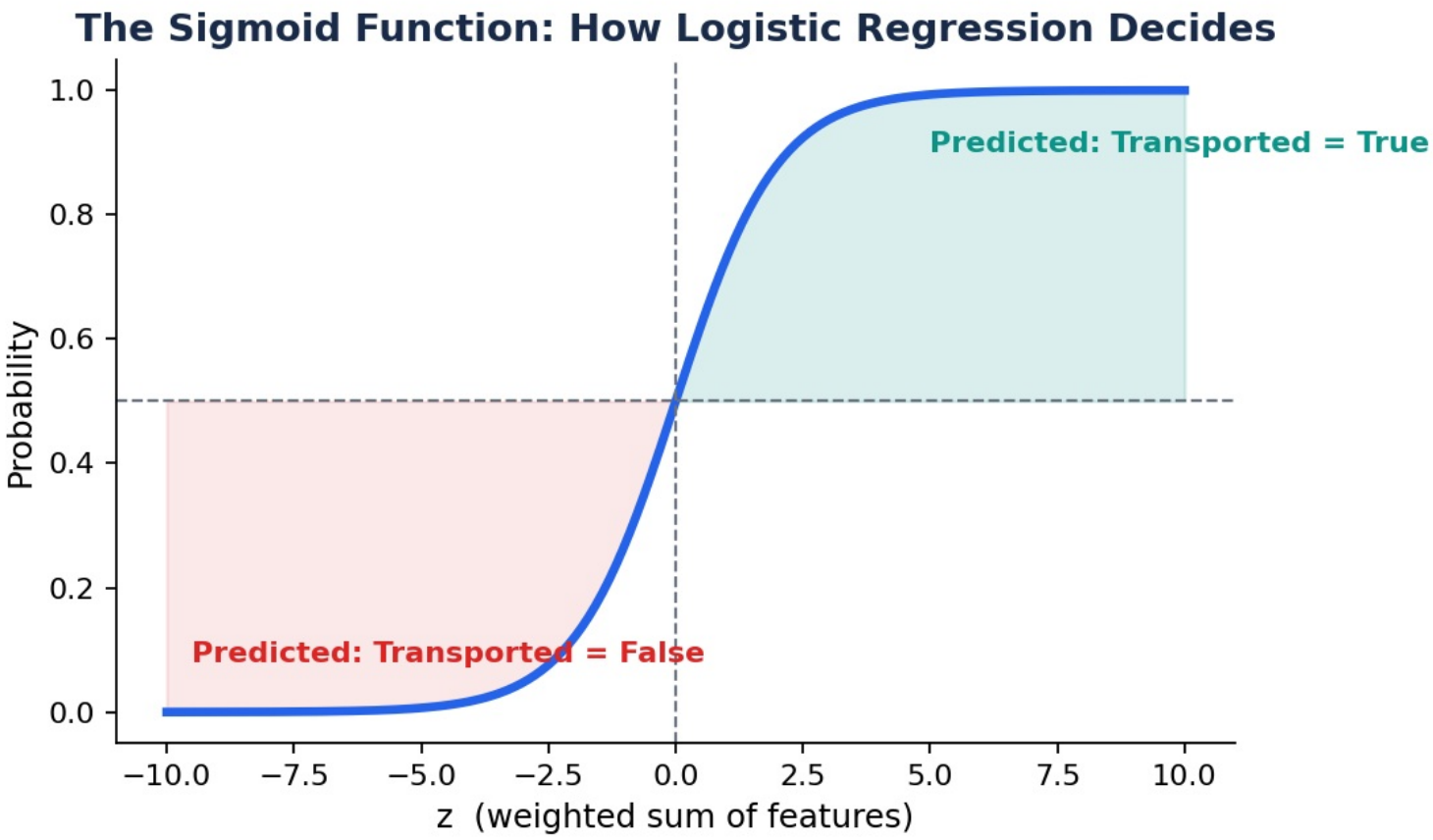
`random_state=42` fixes the "random" shuffle so the split is identical every time the code re-runs — reproducibility, not a magic number.

7. Model 1 — Logistic Regression (the Baseline)

Why train a "weak" model first? Every project needs a floor score. Without a baseline, you can't tell if a fancier model is actually helping or just performing at chance level with extra complexity. (This mattered directly later — see Section 8.2, where Random Forest's first attempt barely beat this baseline.)

How Logistic Regression actually works

Despite the name, it's a **classification** algorithm. It computes a weighted sum of the input features, then squashes that number through the **sigmoid function** into a probability between 0 and 1. Above 0.5 → predict True; below → predict False.




```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

numeric_cols = ['Age','RoomService','FoodCourt','ShoppingMall','Spa','VRDeck','Total Spend']
scaler = StandardScaler()
X_train_scaled = X_train.copy(); X_val_scaled = X_val.copy()
X_train_scaled[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_val_scaled[numeric_cols]   = scaler.transform(X_val[numeric_cols])   # NOT fit — see note below

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train)
accuracy_score(y_val, log_model.predict(X_val_scaled))   # → 0.7798
```

The convergence problem we actually hit

> **What happened:** training initially threw `ConvergenceWarning: lbfgs failed to converge`. Logistic Regression trains by iteratively nudging its internal weights to reduce error, one small step at a time — each step is one "iteration." `max_iter` caps how many nudges it's allowed before giving up.

Root cause: our numeric columns lived on wildly different scales — Age (0-79) vs Total Spend (into the thousands). The optimizer struggled to take balanced steps across such different magnitudes.

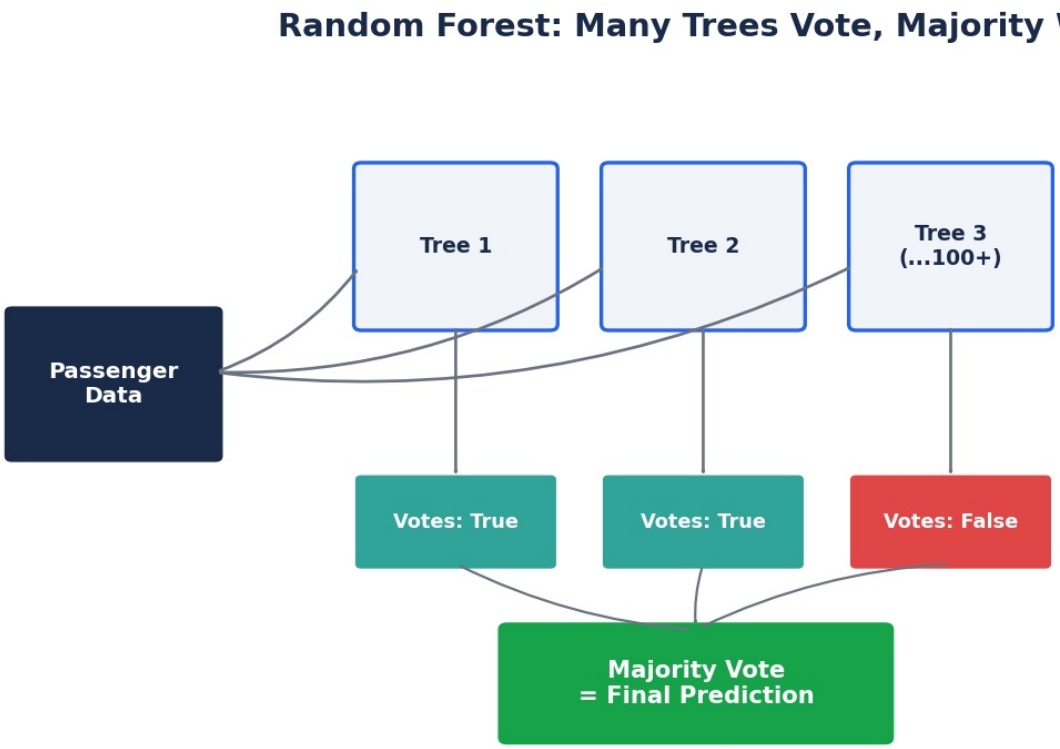
Real fix — feature scaling, not just raising max_iter: `StandardScaler` transforms every numeric column to mean 0, standard deviation 1, so all features sit on a comparable scale. Critically: `.fit_transform()` on training data only, then plain `.transform()` on validation — fitting on validation too would leak information from data the model should never "see" during training decisions.

Result: Logistic Regression baseline locked in at **77.98% validation accuracy** — the number every later model had to beat.

8. Model 2 — Random Forest

How Random Forest actually works

A Random Forest builds many individual decision trees (each trained on a random subset of data and features), and lets them **vote**. The majority vote becomes the final prediction. This "wisdom of crowds" approach makes the ensemble far less likely to be wrong than any single tree.



8.1 First attempt — default settings

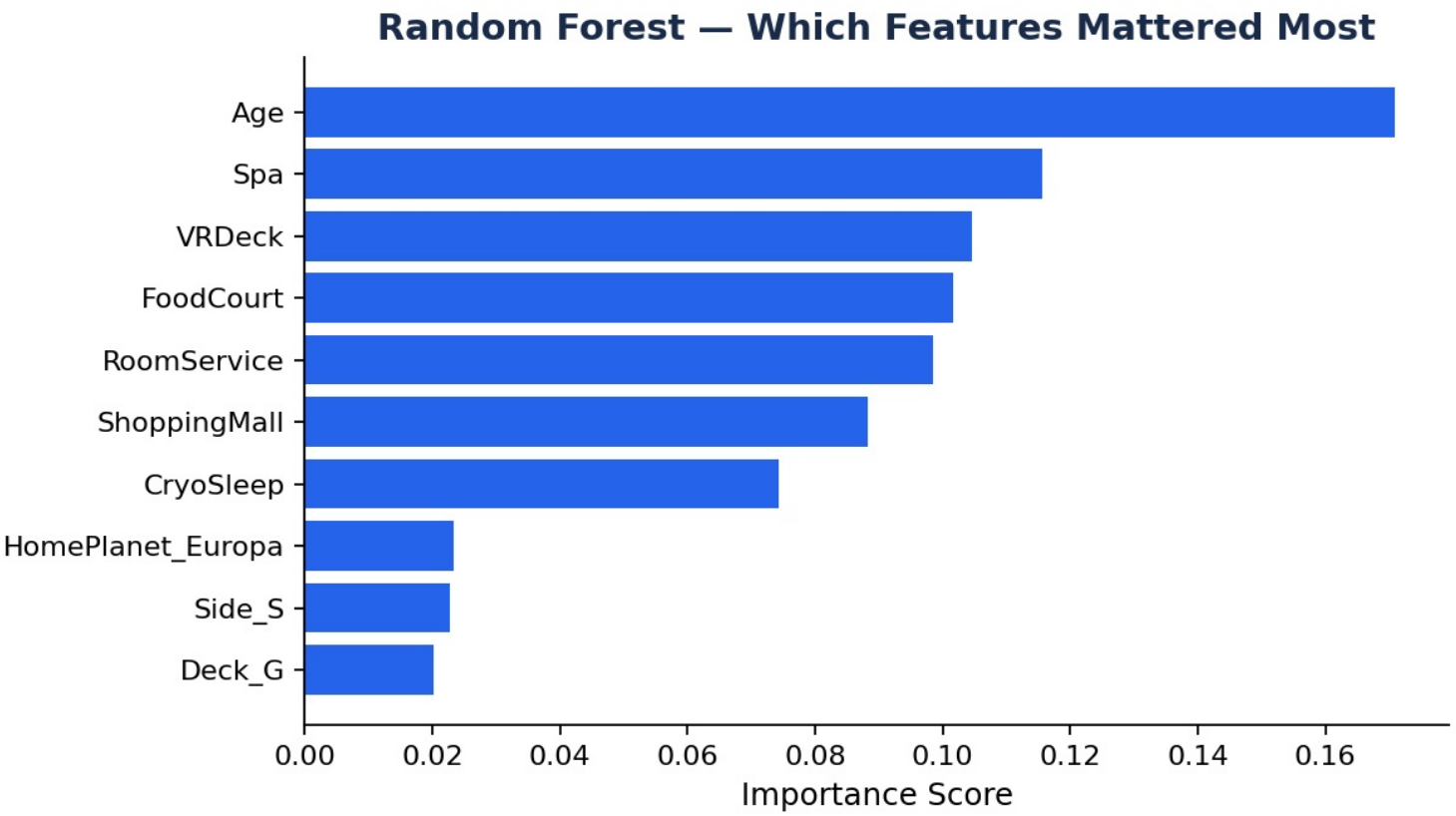
```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)      # NOTE: raw X_train, no scaling needed for trees
accuracy_score(y_val, rf_model.predict(X_val))   # → 0.7826 — barely beat the baseline!
```

Why no scaling for tree-based models: trees split data on thresholds ("is Age > 30?"), not distances or magnitudes — wildly different feature ranges don't confuse them the way they confused Logistic Regression's optimizer.

This result was a warning sign, not a dead end: Random Forest is supposed to outperform a simple linear model. Tying the baseline (0.7826 vs 0.7798) meant something was off — and having the baseline number to compare against is exactly what surfaced this problem instead of it going unnoticed.

8.2 Diagnosing the problem — feature importance

```
importances = pd.Series(rf_model.feature_importances_, index=X_train.columns)
importances.sort_values(ascending=False).head(10)
```



What this revealed: `Total Spend` ranked among the top features — but it's literally just the sum of the 5 individual spend columns sitting right beside it in the same feature list. The model was being handed the same signal twice, diluting its ability to find clean splits.

```
X_train_v2 = X_train.drop(columns=['Total Spend'])
X_val_v2   = X_val.drop(columns=['Total Spend'])
# Result: 0.7763 — barely moved. Redundancy wasn't actually the main bottleneck.
```

8.3 The real fix — hyperparameter tuning against overfitting

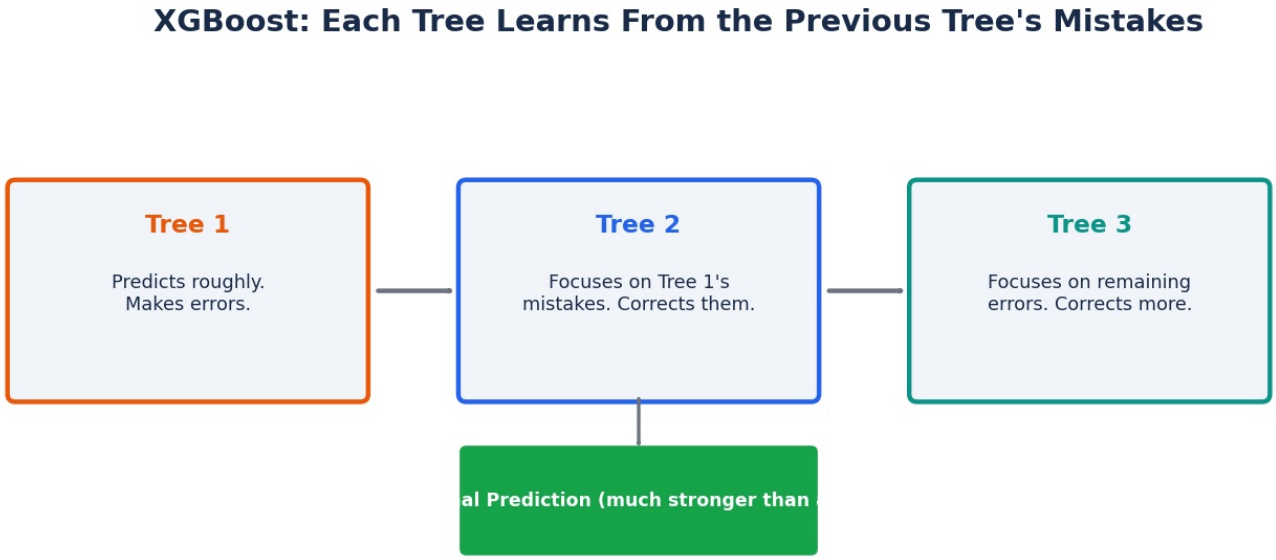
```
rf_model = RandomForestClassifier(
    n_estimators=300,      # more trees voting → more stable predictions
    max_depth=10,          # caps how deep each tree can grow
    min_samples_leaf=5,    # each final decision must represent ≥5 real passengers
    random_state=42
)
# Result: 0.7872 — a real, meaningful jump
```

Why this worked: an untuned Random Forest can grow trees until they perfectly memorize training data — including noise, not just real patterns. That's **overfitting**: great training performance, mediocre validation performance. Limiting `max_depth` (how deep a tree can grow) and `min_samples_leaf` (minimum passengers per final decision) forces trees to learn generalizable rules instead of memorizing individual rows.

9. Model 3 — XGBoost

How XGBoost (gradient boosting) actually works

Unlike Random Forest (many independent trees voting), XGBoost builds trees **sequentially** — each new tree is trained specifically to correct the mistakes of the trees before it. This "boosting" approach often outperforms Random Forest, especially on structured/tabular data like this.




```
from xgboost import XGBClassifier
xgb_model = XGBClassifier(
    n_estimators=300, max_depth=5, learning_rate=0.05,
    random_state=42, eval_metric='logloss'
)
# First attempt result: 0.7844
```

Why these starting parameters, not defaults

Parameter	What it controls	Why this value
max_depth	How deep each individual tree grows	Kept shallower (5) than Random Forest's 10 — since XGBoost trees build on each other sequentially, deep trees risk overcorrecting/overfitting fast
learning_rate	How much each new tree is allowed to influence the final prediction	Lower (0.05) = slower, more careful learning; needs more trees to compensate, hence higher n_estimators
n_estimators	Number of sequential trees built	300 — compensates for the low learning rate, more trees = more correction rounds

10. Hyperparameter Tuning with RandomizedSearchCV

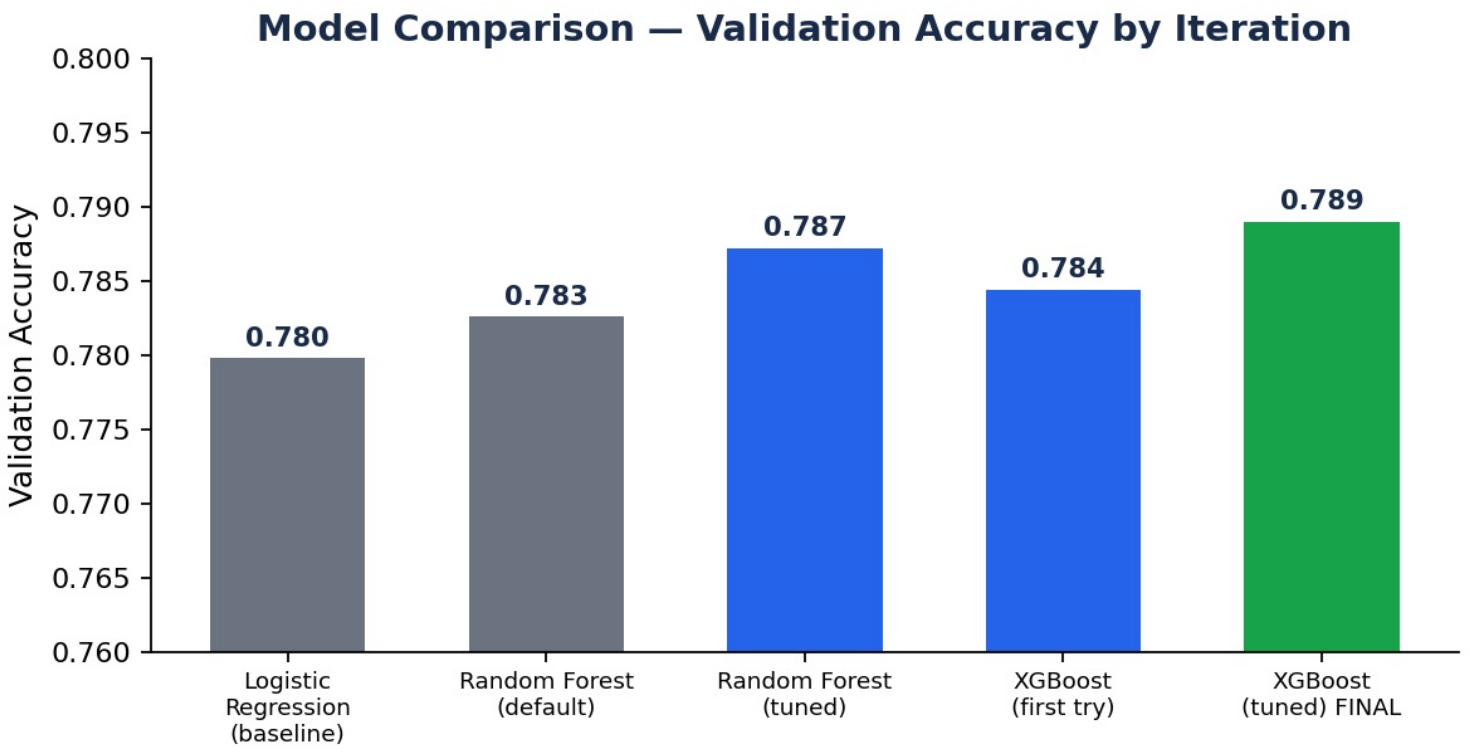
Why not keep hand-tuning one parameter at a time: it's slow, unsystematic, and easy to miss good combinations. RandomizedSearchCV tries many parameter combinations automatically and tells you the best one — a genuinely more rigorous, defensible approach for an interview to describe.

```
from sklearn.model_selection import RandomizedSearchCV
param_grid = {
    'n_estimators': [200, 300, 400], 'max_depth': [3, 4, 5, 6],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0], 'colsample_bytree': [0.8, 1.0]
}
search = RandomizedSearchCV(
    XGBClassifier(random_state=42, eval_metric='logloss'),
    param_distributions=param_grid, n_iter=20, cv=3,
    scoring='accuracy', random_state=42, n_jobs=-1
)
search.fit(X_train_v2, y_train)
# Best params: n_estimators=400, max_depth=6, learning_rate=0.01, subsample=0.8, colsample_bytree=1.0
# Best cross-validated score: 0.8077
```

What cross-validation (cv=3) adds: instead of testing each parameter combination on a single train/validation split (which can get lucky or unlucky), the training data is split into 3 chunks; the model trains/tests 3 times rotating which chunk is held out, then the results are averaged. Far more reliable than a single split's score.

`n_iter=20` means 20 random combinations tried (not every possible combination — checking all of them would take far too long); `n_jobs=-1` uses all available CPU cores to speed this up.

11. Model Comparison & Final Choice



Model	Validation Accuracy	Notes
Logistic Regression (baseline)	0.7798	Needed feature scaling to converge
Random Forest (default)	0.7826	Barely beat baseline — signal it was overfitting
Random Forest (tuned)	0.7872	max_depth + min_samples_leaf fixed overfitting
XGBoost (first attempt)	0.7844	Untuned starting point
XGBoost (RandomizedSearchCV tuned)	**0.7890**	**Final model — best result**

Why XGBoost tuned won, in one sentence for an interview: "I established a linear baseline, found my first tree-based model wasn't actually beating it — which told me it was overfitting — fixed that with depth/leaf constraints, then used cross-validated randomized search to systematically tune a gradient-boosted model, which ultimately generalized best on held-out data."

12. Predicting on Test Data & Submission

> **The golden rule when cleaning test data:** every statistic used to fill missing values (median, mode) must come from **training data only**, never recalculated fresh from test data. This is the same "don't leak information" principle as the StandardScaler, applied to the entire cleaning pipeline.

```
# Age example — reuse TRAIN's median, don't recompute from test:
test_df['Age'] = test_df['Age'].fillna(train_df['Age'].median())

# Column alignment — one-hot encoding test data independently can create mismatched columns
# (e.g. if no test passenger happens to be on Deck T, that column won't exist in test_df)
test_df_aligned = test_df.reindex(columns=X_train_v2.columns, fill_value=0)

final_test_preds = final_model.predict(test_df_aligned)
submission = pd.DataFrame({
    'PassengerId': test_passenger_ids,
    'Transported': final_test_preds.astype(bool)
})
submission.to_csv('submission.csv', index=False)
```

13. Key Interview Talking Points

- **"Why did you use median instead of mean for Age?"** — Mean (28.83) and median (27.0) differed slightly, indicating mild right-skew. Median is robust to outliers, so it better represents a "typical" passenger for imputation.
- **"Why one-hot encoding instead of label encoding for HomePlanet?"** — Label encoding (0,1,2) would imply a false numeric ordering between categories that don't have one. One-hot avoids that.
- **"How did you handle missing CryoSleep values?"** — Used domain logic: CryoSleep passengers can't spend money, so zero total spend strongly implies CryoSleep=True. Validated the assumption on the actual missing-data subset before applying it, rather than assuming it would hold.
- **"Why did your first Random Forest disappoint you?"** — It barely beat a simple Logistic Regression baseline, which signaled overfitting on default settings. Diagnosed via feature importance, then fixed with max_depth and min_samples_leaf constraints.
- **"How did you choose your final model?"** — Compared model families (linear vs bagging vs boosting) rather than committing to the first one that worked, and used cross-validated RandomizedSearchCV for principled hyperparameter tuning rather than manual guessing.
- **"How did you avoid data leakage?"** — Fit scalers and computed fill statistics only on training data; applied (never re-fit) those same values to validation and test sets.

14. Full Code Appendix

The complete, runnable notebook code — cleaning, feature engineering, all three models, tuning, and submission generation — is included in the accompanying GitHub repository (see README.md and Project_Report.pdf for the model-selection summary).